# CSCE 608 Project Report

Vivian Vo, UIN: 528004304

March 20th, 2026

## Part A. Project Description

For the project, I am proposing on creating an **Art Gallery Management Database** as the domain. This database serves to help an art gallery's daily operations, such as cataloging. This includes tracking inventory of different artworks, which artwork was created why which artist, and which type of artwork or categories seem to be the most popular or are valued the highest?

As typically expected of an art gallery, galleries are dedicated spaces meant to house and showcase different types of artwork or art pieces. **Artwork** can consist of various media, including: paintings, drawings, sculptures, carvings, engravings, photographs, etc.

Each art piece is credited to an **Artist**. An artist can have many art pieces to their name, work in various mediums, and the price of the work. Artwork is also classified by some **Category** like the type of materials used to create it (e.g., oil on canvas, acrylic on canvas, watercolors, gouache, etc), the location of the art, the "theme" or subject of the art (e.g., impressionist movement, 13th century). **Images** of these artworks are also shown.

Galleries also hold **Exhibitions**, which are showcases meant to display specific art. Exhibitions can highlight a lone artist's work; they can display works in a specific art movement; works in a certain era or decade; works done by various cultures, genders, and races; and works based on a *theme or subject* (e.g., florals, American romanticism, black artists, etc). Here, another service the database can provide is curation, which helps art gallery owners make these organizational decisions over the pricing of each art piece or how many will go in the exhibition, what theme an exhibition has, and when the exhibition will take place.

**Customers** attend galleries, and galleries keep track of their loyal subscribed customers. This service can be described as customer management. Art gallery owners will be able to track information like a Customer and the donations they have made to the gallery. For donations, this can be contributed in a monetary form. We also track what purchases a customer has made and how much they paid for an artwork.

Now with this, there poses some potential unique challenges. The first being that **many-to-many** relationships described like **Belongs** or **Displays** can cause potential conflicts due to the nature of many artworks belonging to many categories and many artworks being displayed in many exhibitions over time. This could conflict with trying to keep the entities or relations in Boyce-Codd Normal Form (BCNF) in order to prevent redundancy.

Some minor details to track is in the **Customer** and **Artwork** entities, we should track who purchased what, and if an artwork has been purchased. This information should dynamically update whenever a purchase is made the listing for the artworks availability should also be updated accordingly.

## Part B. E/R Diagram

In total the **main entities** and their attributes described from **Part A** are:

1. **Artist**: This entity contains the name of the artist, their birthplace, the time period it was created, and the type of art (e.g., painting, statue, etc).

   - artistID (PK): INT,
   - artistName: VARCHAR(256),
   - birthplace: VARCHAR(256),
   - timePeriod: VARCHAR(32),
   - artType: VARCHAR(64),

2. **Artwork**: This entity contains art pieces, and lists attributes such as the title of the art piece, type of art piece (e.g., print, statue, bust, etc), its medium (e.g., oil on canvas) the name of the artist who created the piece, the year it was created, the price of the art, and if it is available for purchase.

   **Attributes**:

   - artworkID (PK): INT,
   - title: VARCHAR(2048),
   - creationYear: VARCHAR(256),
   - artType: VARCHAR(64),
   - medium: VARCHAR(2048),
   - price: DECIMAL(15,2),
   - canPurchase: VARCHAR(32):

3. **Images**: This is a simple entity with two attributes. It serves to store a bunch of images associated with an artwork separately via a URL. It is structured this way via how the National Gallery of Art (NGA)'s Open Data repository sorts data.

   **Attributes**:

   - uuID (PK): VARCHAR(64),
   - imageURL: VARCHAR(512),
   - imageObjectID: INT

4. **Customers**: This entity lists the customer name, their address, and the amount of donations they made.

   - customerID (PK): INT,
   - customerName: VARCHAR(256),
   - address: VARCHAR(256),
   - donationAmount: DECIMAL(15, 2),
   - donationDate: TIMESTAMP

5. **Exhibition**: This entity lists the Exhibition name, the theme or subject of the exhibition, the art pieces being displayed, and the start and end date of the exhibition.

   - exhibitionID (PK): INT,
   - exhibitionTitle: VARCHAR(2048),

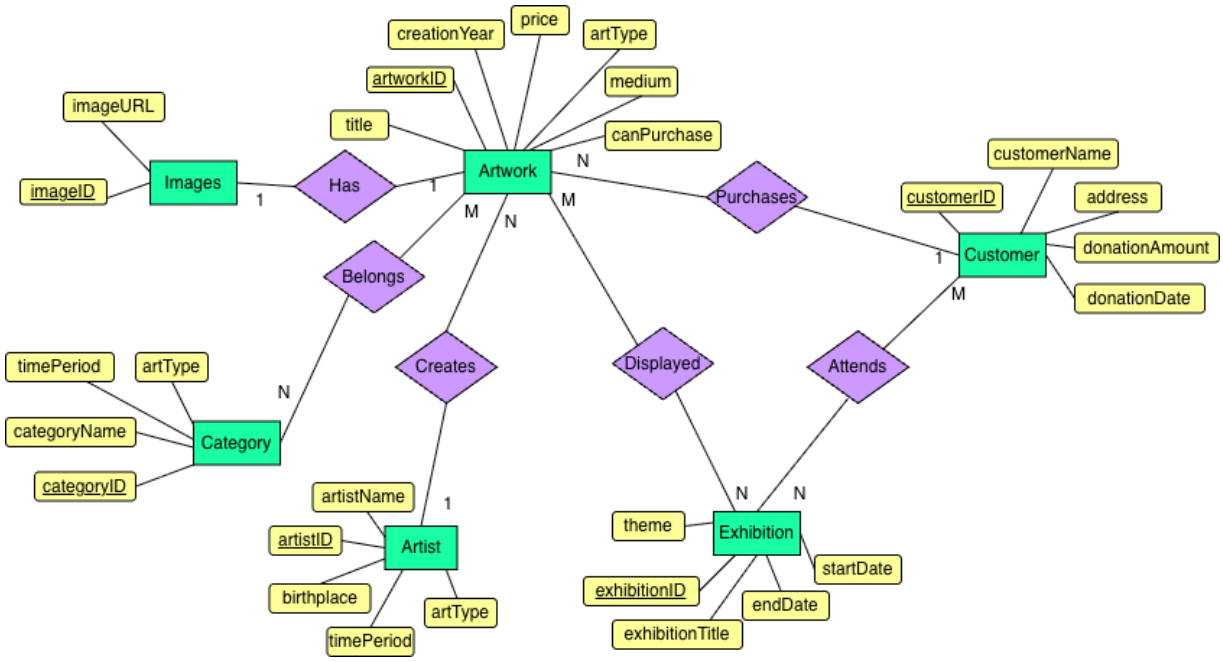


Figure 1: E/R Diagram

- theme: VARCHAR(100),
- startDate: DATE,
- endDate: DATE

6. **Category**: This entity lists the types of classifications of the art piece like the art type (e.g., print, statue, bust, etc).
   - categoryID (PK): INT,
   - categoryName: VARCHAR(100),
   - timePeriod: VARCHAR(32),
   - artType: VARCHAR(64)

All of these entities and their attributes were pulled directly from what I described in **Part A**. I described the different aspects of an art gallery, which translate to the entities. I describe what each entity needed to function or what information it needed to track, which became the attributes. From there, I explained some common services or functions that can be performed by an entity with its attributes or between two entities. These services or functions became the relationships between the entities.

Some **entity relationships** include (the diamond shapes from the **Figure 1**).:

1. ***Creates* (Artist ↔ Artwork)**: This is a **one-to-many** relationship. One artist can have many different artworks to their name, but an artwork can only have been created by one artist.

2. ***Has* (Artwork ↔ Images)**: This is a **one-to-one** relationship. One artwork has exactly one image associated with it and vice versa.

3. ***Displayed*** **(Exhibition ↔ Artwork)**: This is a **(many-to-many)** relationship. An exhibition can store many different types of artworks, and an artwork can be displayed in different exhibitions.
4. ***Belongs*** **(Artwork ↔ Category)**: This is a **(many-to-many)** relationship. An artwork can be classified in many different categories (e.g., oil on canvas, impressionist) while a category can list, many different artworks.
5. ***Purchases*** **(Customer ↔ Artwork)**: This is a **(one-to-many)** relationship. A single customer can purchase many pieces of art, but an artpiece can have been purchased by only a single customer.
6. ***Attends*** **(Customer ↔ Exhibition)**: This is a **(many-to-many)** relationship. A customer can attend many different exhibitions, while an exhibition can be visited by many customers.

# Part C. Table Normalization

## Relations

Here we translated the entities from the E/R diagram into a relational database schema. Primary keys are denoted by **PK** while foreign keys are listed as **FK**. For relationships that are one-to-many, foreign keys are utilized to handle the "many" side of the relationship. For many-to-many relationships, this is handled by creating separate tables.

1. **Artists** (artistID (PK), artistName, birthplace, timePeriod, artType)
2. **Artwork** (artworkID (PK), title, creationYear, artType, medium, price, canPurchase)
   - FK: artistID will be used to reference the Artist entity. This is for the relationship **Creates**.
   - FK: customerID will be used to reference the Customers entity. This is for the relationship **Purchases**.
3. **Images** (uuID (PK), imageURL, imageObjectID)
4. **Customers** (customerID (PK), customerName, address, donationAmount, donationDate)
5. **Exhibitions** (exhibitionID (PK), exhibitionTitle, theme, startDate, endDate)
6. **Category** (categoryID (PK), categoryName, timePeriod, artType)
7. **Displays** (artworkID, exhibitionID)
8. **Belongs** (artworkID, categoryID)
9. **Attends** (customerID, exhibitionID)

## Non-trivial Functional Dependencies

This section lists all the non-trivial functional dependencies (FD) of all the relations. If a relation has no non-trivial FD's, then the attributes together form the key.

1. **Artists**: $artistID \rightarrow \{artistName, birthplace, timePeriod, artType\}$
2. **Artwork**: $artworkID \rightarrow \{title, creationYear, artType, medium, price, canPurchase, artistID, customerID\}$
3. **Images** $uuID \rightarrow \{imageURL, imageObjectID\}$

4. **Customers**: $customerID \rightarrow \{customerName, address, donationAmount, donationDate\}$

   **Other Functional Dependency**:

   - $categoryName \rightarrow \{timePeriod, artType\}$

5. **Exhibitions**: $exhibitionID \rightarrow \{exhibitionTitle, theme, startDate, endDate\}$

6. **Category**: $categoryID \rightarrow \{categoryName, origin, artType\}$

7. **Displays**: $\{artworkID, exhibitionID\}$ has no non-trivial FDs.

8. **Belongs**: $\{artworkID, categoryID\}$ has no non-trivial FDs.

9. **Attends**: $\{customerID, exhibitionID\}$ has no non-trivial FDs.

### Boyce-Codd Normal Form (BCNF)

BCNF rules include that the relation 1) satisfies Third Normal Form (3NF) and 2) for every non-trivial FD $X \rightarrow Y$, $X$ is a superkey, which is a set of one or more columns that can uniquely identify a row in a table. After going through the normalization process, we can see that there was little to no change in the table structures, save for the Category entity.

Let's say for example, we get a specific category name like "Renaissance" which points to a time period and art style, then this makes **categoryName** a determinant; however, not a superkey. Still this violates BCNF due to a dependency existing ona non-key attribute.
Assume the dependency $categoryName \rightarrow \{timePeriod, artType\}$ exists, we decompose it.

**Decompose on *categoryName***:

- **Relation 1**: $(categoryName, timePeriod, artType)$
- **Relation 2**: $(categoryID, categoryName)$

We can no longer further decompose either relation as they are non-trivial dependencies. We end up with the **Relations 1** and **Relations 2** as a result.

As for the other main entities all have primary keys that act as determinants, they are effectively all candidate keys, which meet the requirements to be in BCNF. The other tables, such as **Displays, Belongs**, and **Attends**, are all key relations, meaning that since there are no non-trivial FDs, they satisfy BCNF. This is why the table structure remains the same.

## Part D. Data Collection

### Data Information & Transformation

The data for this project was primarily sourced from the National Gallery of Art (NGA)'s Open Data GitHub repository. Important attribute information was garnered from a handful of CSV files. From the objects.csv file, I was able to grab data such as the artwork id's, titles, place of origin and more to populate the Artwork relation. Little was needed to edit upon it asides from generating fake prices. Data for artists and common customers (who served as our donors) were pulled from the constituents.csv file, while exhibition titles and themes were derived from objects_text_entries.csv. Since the raw data represented a the interactions, the objects_constituents.csv file was utilized as a relationship table to map the many-to-many relationships between artists and their artworks.

To ensure the uniqueness of key attributes and maintain integrity, the data ingestion process utilized PostgreSQL primary keys on unique identifiers such as artworkid and constituentid. During the data cleaning and prepping phase, missing or duplicate data was either filled in by a generated value using the Python Faker library and the uuid module or was removed in general (if a duplicate). This allowed for the creation of a generated Customer table with unique identifiers, preventing record collisions during the population. Because of this I was able to perform some JOIN opereations, such as a join between the Customer, Artwork, and Exhibition tables. This allowed for achieving complex analytical information, such as identifying "Top Customers", the top donors in the gallery, or most common artwork categories.

# Part E. User Interface

The project's databases was built using PostgreSQL then hosted via Neon.tech database. Software packages like python's framework, Streamlit, were used to construct a simple web page for our database features. The primary user, being an art gallery owner, is able to navigate the webapp in several ways. First a navigation (nav) bar with five different pages: Gallery, Analytics, Exhibitions, Customers, and Manage.

## How to Navigate Page

You are a gallery owner; these are the features you can view.

- Follow Navbar to different pages
- Search bars are labeled as such. In the Gallery, search for any artwork. In Exhibitions, search for the exhibit name or theme. Here, you can view how many artworks are displayed, how many customers attended, and how much funding was put into the exhibit.
- Analytics page only needs to scroll. This aggregates the top categories of artwork, the amount for each category, and their cost totaled together.
- Customers is a scrolling table as well. It lists all customers in the gallery and how much they've donated. It is sorted in descending order.
- The Manage page is where a gallery owner would search for an artwork based on its ID and alter its price.

The webapp has a few functions that can both retrieve and update information. Each function related to a specific SQL query:

- **Gallery Display**: An art gallery owner can search the entire artwork inventory by it's title. This is implemented via a SELECT query with a LEFT JOIN between the Artwork and Images table, and specific case-insensitive filtering such as lowercase or just some keywords or letters.
- **Inventory and Category Stats**: An art gallery owner will be able to view the gallery's inventory value or the most common categories in their inventory, and how much each total to. It is implemented using GROUP BY and aggregate functions (COUNT, SUM) to calculate the individual "Count" and "Total Value" across different art categories.
- **Exhibition Search:** An art gallery owner can view their exhibits and how many customers have attended. The SQL used are a three LEFT JOINs between Exhibition, Displays, and Attends to aggregate the count of artworks to an exhibition and number of customers visited per exhibit.

- **Listing Customer Information**: An art gallery owner can view the gallery's "Top Customers". This is done by first creating a sub-query where the HAVING clause filters for customers whose total donations exceeds the average (AVG) spending of all customers.
- **Managing Artwork Pricing** (Manage): An art gallery owner can update artwork prices through searching for specific artwork id's. This is done via a simple UPDATE statement.

## Part F. Project Source Code

- **GitHub**: https://github.com/voviv2000/art-gallery-database
- **Webapp URL**: https://vivian-art-gallery-tamu-9131538c5461.herokuapp.com/

## Part G. Discussion

During my time developing this database, I found that even though carefully thinking about the entities and their relationships before hand, when implementing the project via coding and SQL queries, more changes had to be to made that I did not foresee. Such as the addition of my Images table because I forgot to include it in my original table, only to find out that NGA's repository stores their images separately. This made it so I had to make an entirely separate table to link to each artwork and their artwork image's URL. This was important because other information or attributes of an image like width and height, had to be tracked as well.

Other difficulties that arose was trying to use the given atrributes from NGA's CSV files, such as the the customerIDs. This was stored as a VARCHAR in NGA's system but for some reason it made it difficult populate my database. I guess there were a lot of comma or quotes so I opted for a more simple solution of generating a numeric integer id for the customers instead and mapped them manually myself. Other things like the SQL queries were not that difficult, most were simple aggregate commands involving GROUP BY, HAVING clauses, or multiple JOIN statements when involving multiple entities.

What I learned overall was that database design takes a lot of planning and forethought because even though I believed I thought of every entity needed, their relations to one another and what queries I could draw from each, it was not as easy as I expected. When actually implementing the information, more details pop up and you find that your original schema is not the most efficient so you must make changes. Either way, I have a better understanding and appreciation of how detailed and thorough database design must be.